Probabilistic Reasoning: Bayesian Networks

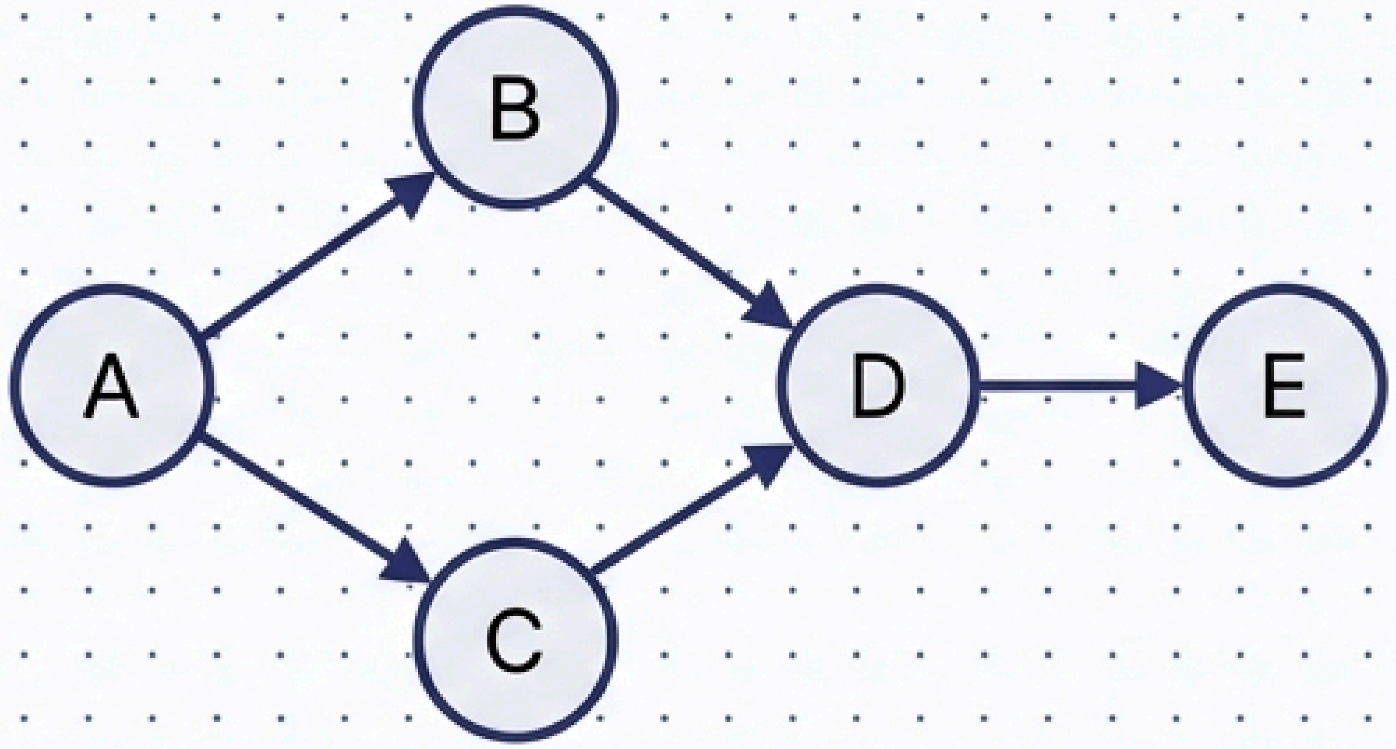
The Ultimate Exam Blueprint
(Chapters 13.1 – 13.4 + Laplace Smoothing)

Module 1:	Module 2:	Module 3:	Module 4:
Syntax & Problem-Solving	Smoothing & Semantics	Independence & D-Separation	Exact & Approximate Inference Algorithms

The FJPD Trap

- Requires specifying probabilities for every possible world.
- Unnatural and tedious to build.
- **Complexity:** Exponential growth $O(2^n)$.
- **Example:** 30 Boolean variables = over 1 billion probabilities.

Bayesian Networks



- Exploits conditional independence to create a locally structured (sparse) system.
- **Complexity:** Linear growth.
- **Example:** 30 nodes with max 5 parents = 960 probabilities.

Key Insight: A Bayesian Network represents the **EXACT** same joint distribution as the FJPD but fundamentally solves the $O(2^n)$ memory and calculation bottleneck

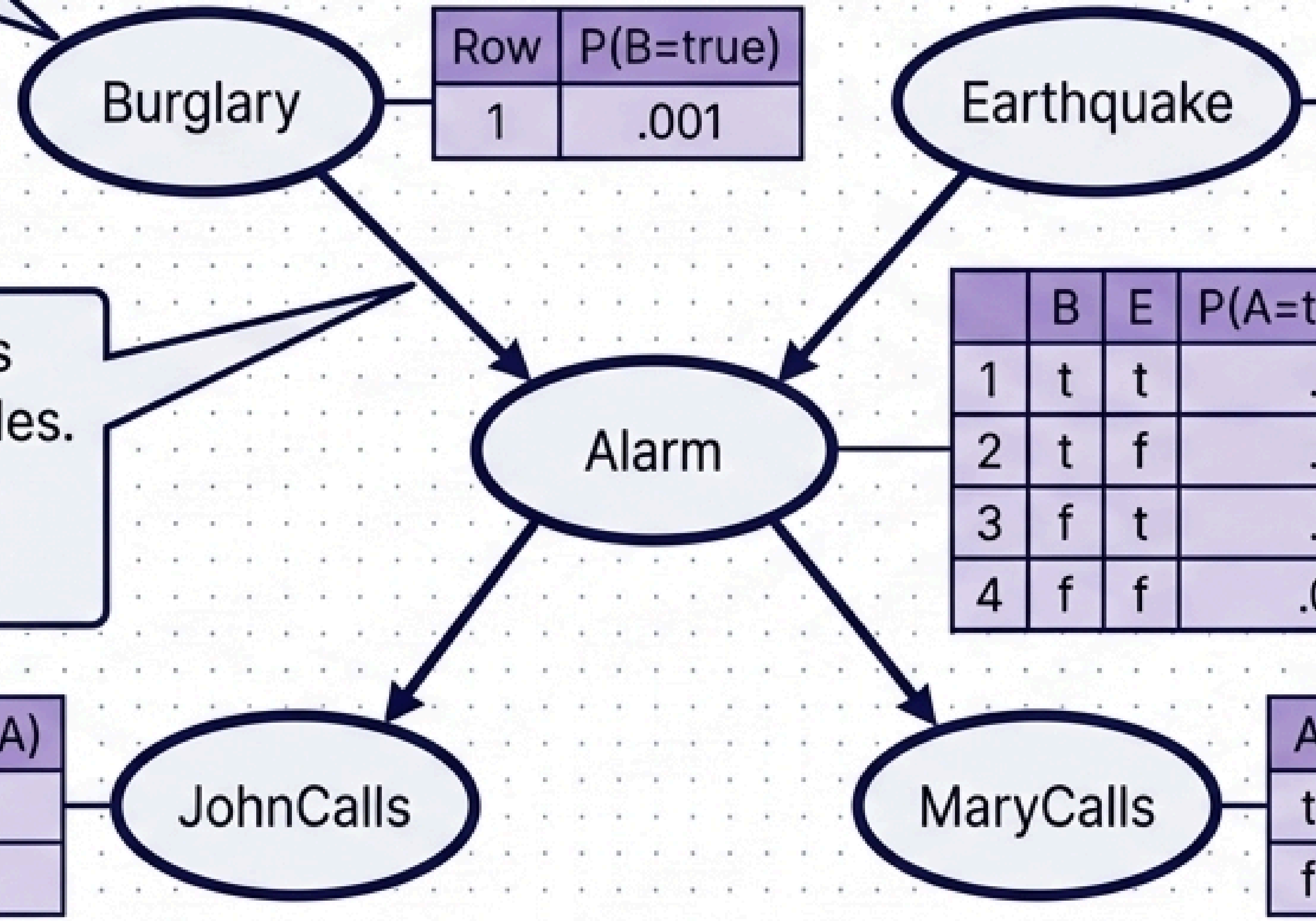
Anatomy of a Bayesian Network

1 **Nodes:** Random variables (discrete or continuous).

2 **Directed Edges:** Arrows connecting pairs of nodes. If $X \rightarrow Y$, X is a direct influence (parent) of Y.

3 **DAG:** Directed Acyclic Graph. The topology contains zero directed cycles.

4 **CPT:** Conditional Probability Table. Quantifies the effect of parents: $P(X_i \mid \text{Parents}(X_i))$. Each row is a conditioning case summing to 1.



Row	P(B=true)
1	.001

Row	P(E=true)
1	.002

	B	E	P(A=true|B,E)
1	t	t	.95
2	t	f	.94
3	f	t	.29
4	f	f	.001

A	P(J=true|A)
t	.90
f	.05

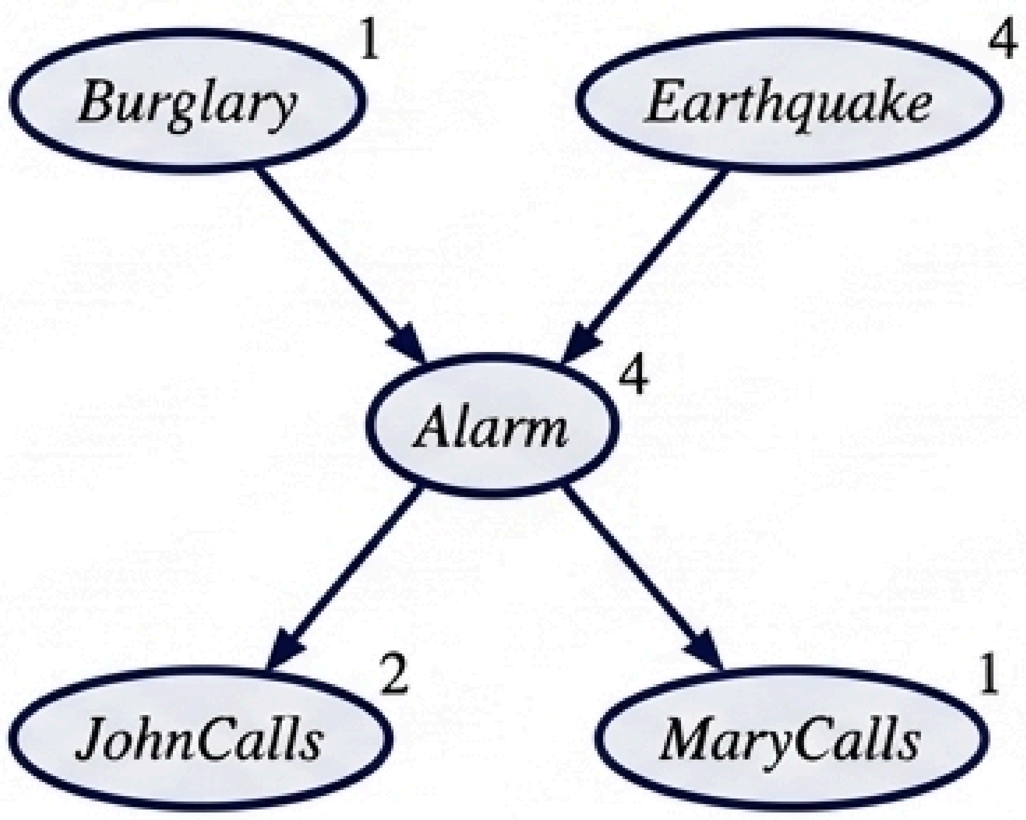
A	P(M=true|A)
t	.70
f	.01

Exam Tip: The topology alone defines conditional independence; the CPTs provide the local quantitative paramet

Constructing a BN & The Trap of Variable Ordering

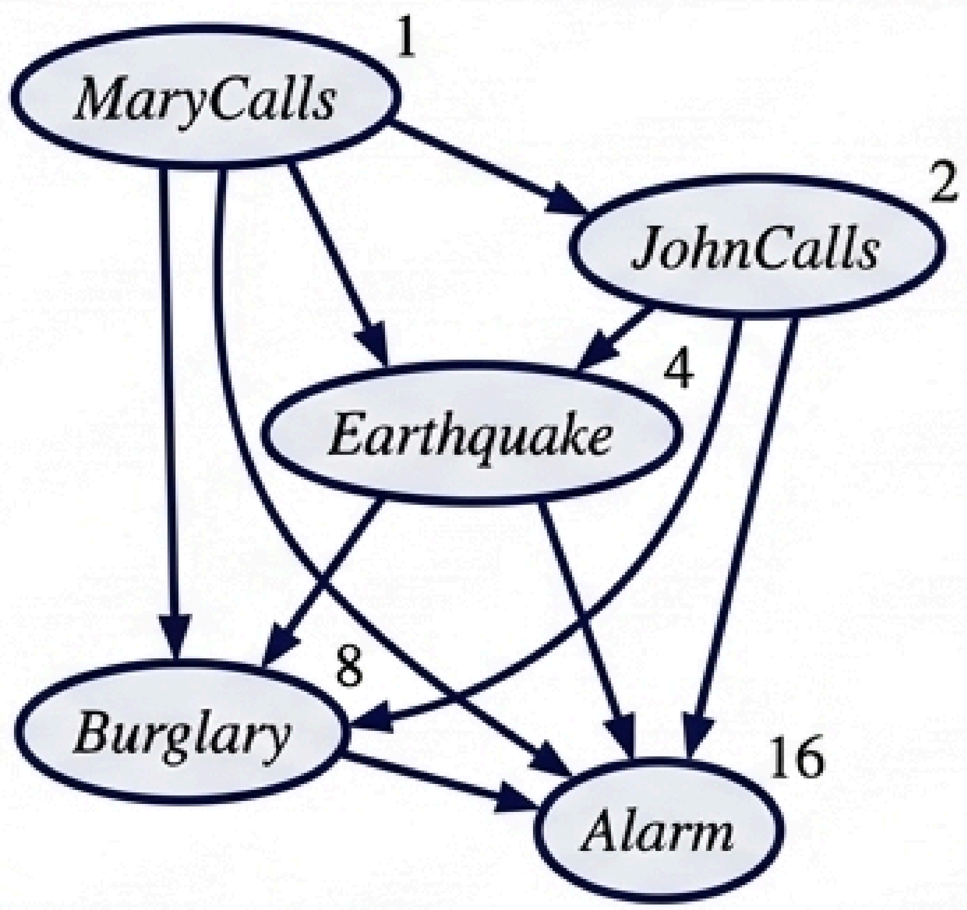
The Construction Algorithm: 1. Determine variables. 2. Order them (Crucial). 3. For `i=1` to n: Choose minimal parents, insert links, define CPT.

Causal Ordering (Causes → Effects)



- **Order:** B, E, A, J, M
- **Result:** Clean, sparse graph. Requires only 10 parameters.
- Natural and easy to assess.

Diagnostic Ordering (Effects → Causes)



- **Order:** M, J, E, B, A
- **Result:** Cluttered network. Requires 31 parameters.
- Forces unnatural probability judgments.

Exam Tip: Always order variables chronologically or causally. Bad ordering ruins the local structure and forces the network back toward $O(2^n)$ complexity.

Fixing the Zero-Probability Trap: Laplace Smoothing

The Zero-Probability Trap

If an event is **never observed in training data** (e.g., **0** false alarms in **10 days**), classical probability assigns it **0%**.

In a Bayes Net or Naive Bayes, a **single 0 wipes out the entire joint probability** due to **multiplication.**

The Solution - Exploded Formula

count(x) : observed occurrences

+ 1 : The 'Add-One' smoothing factor (hallucinating one observation of every event)

$$P(x) = \frac{\text{count}(x) + 1}{N + k}$$

N : Total number of observations

k : Number of possible distinct values the variable can take

Numerical Example

Example: 10 coin flips, all Heads.

Without smoothing: `P(Tails) = 0 / 10 = 0`

With smoothing: `P(Tails) = (0 + 1) / (10 + 2) = 1/12 = 0.083`

Insight: Essential when learning CPTs from limited datasets to prevent absolute impossibility.

Semantics: The Chain Rule & Factorization

The Universal Chain Rule

Works for ANY joint distribution.

$$P(x_1, \dots, x_n) = \prod P(x_i \,|\, x_{i-1}, \dots, x_1)$$

Applying BN Conditional Independence
(Each node is independent of predecessors given its parents)

Bayesian Network Factorization

The global joint probability collapses into a product of local CPT lookups.

$$P(x_1, \dots, x_n) = \prod P(x_i \,|\, \text{parents}(X_i))$$

Example Calculation

Example: $\mathrm{P}(\mathrm{j}, \mathrm{m}, \mathrm{a}, \neg\mathrm{b}, \neg\mathrm{e}) = \mathrm{P}(\mathrm{j}|\mathrm{a}) \times \mathrm{P}(\mathrm{m}|\mathrm{a}) \times \mathrm{P}(\mathrm{a}|\neg\mathrm{b}, \neg\mathrm{e}) \times \mathrm{P}(\neg\mathrm{b}) \times \mathrm{P}(\neg\mathrm{e})$

$= 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998$

$= 0.000628$

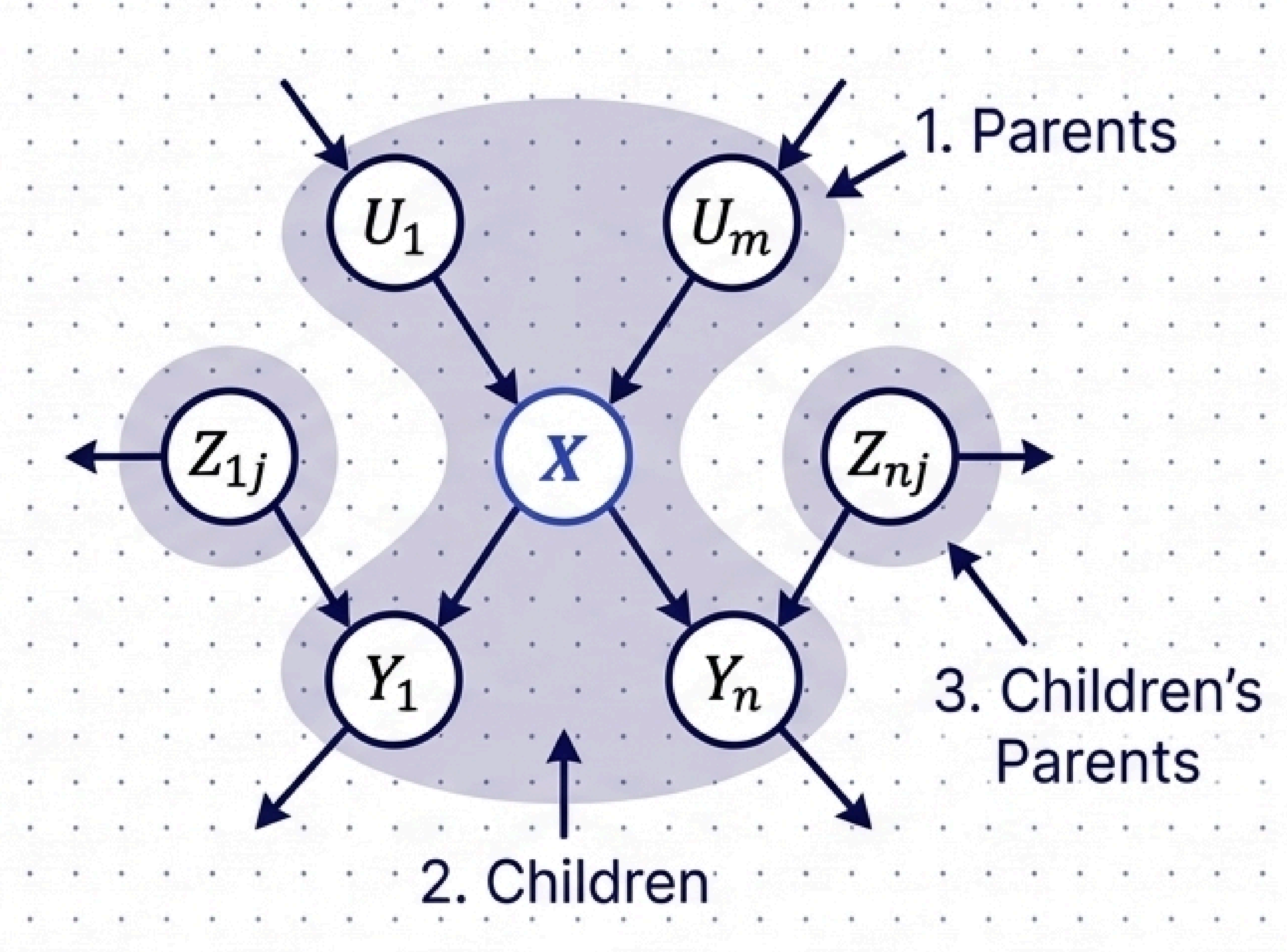
Conditional Independence I: Local Semantics & Markov Blanket

The Non-descendants Property

A variable is conditionally independent of all its non-descendants, given its parents.

The Markov Blanket

A node is conditionally independent of ALL OTHER NODES in the entire network, given its Markov Blanket.



Exam Tip: Why include children's parents? Because 'explaining away' (an effect having two possib causes) creates a dependency loop that must be blocked by knowing the other parent.

Conditional Independence II: D-Separation

Goal: How do we determine if Set X is independent of Set Y, given evidence Set Z? We check if Z "D-Separates" them globally.

Step 1: Ancestral Subgraph

Remove all nodes that are not in X, Y, Z, or their ancestors.

→

Step 2: Moral Graph

Add undirected links between any unlinked pair of nodes that share a common child ("marrying the parents").

→

Step 3: Disorient

Replace all remaining directed arrows with undirected links.

→

Step 4: Check Blocking

If every path between X and Y is blocked by a node in Z, they are D-Separated.

Key Insight: If D-Separated, X is conditionally independent of Y given Z. If not, the original net does not strictly require them to be independent.

Efficient CPT Representations

Reducing the $O(2^k)$ parameter burden in Conditional Probability Tables.

Deterministic Nodes

- Value is specified exactly by parents with zero uncertainty.
- Example: Logical OR, Minimum price, Sum of inflows.

Context-Specific Independence (CSI)

- Independence that only exists for certain values of a parent.
- Example: Car Damage is independent of Ruggedness IF Accident = false.

Noisy-OR (Exam Favorite)

- Generalization of logical OR. Used for diseases/symptoms.
- Reduces CPT from $O(2^k)$ to $O(k)$ parameters.
- Formula: $P(x_i \mid \text{parents}(X_i)) = 1 - \prod q_j$ (Where q_j is the probability the parent fails to cause the effect).

Continuous & Hybrid BNs

- Mixes continuous and discrete variables.
- Handles infinite values via Discretization (intervals).
- Alternatively, defines standard distributions (e.g., Linear-Gaussian where child variance depends on continuous parents).

Exact Inference I: The Enumeration Trap

Goal & Mechanics

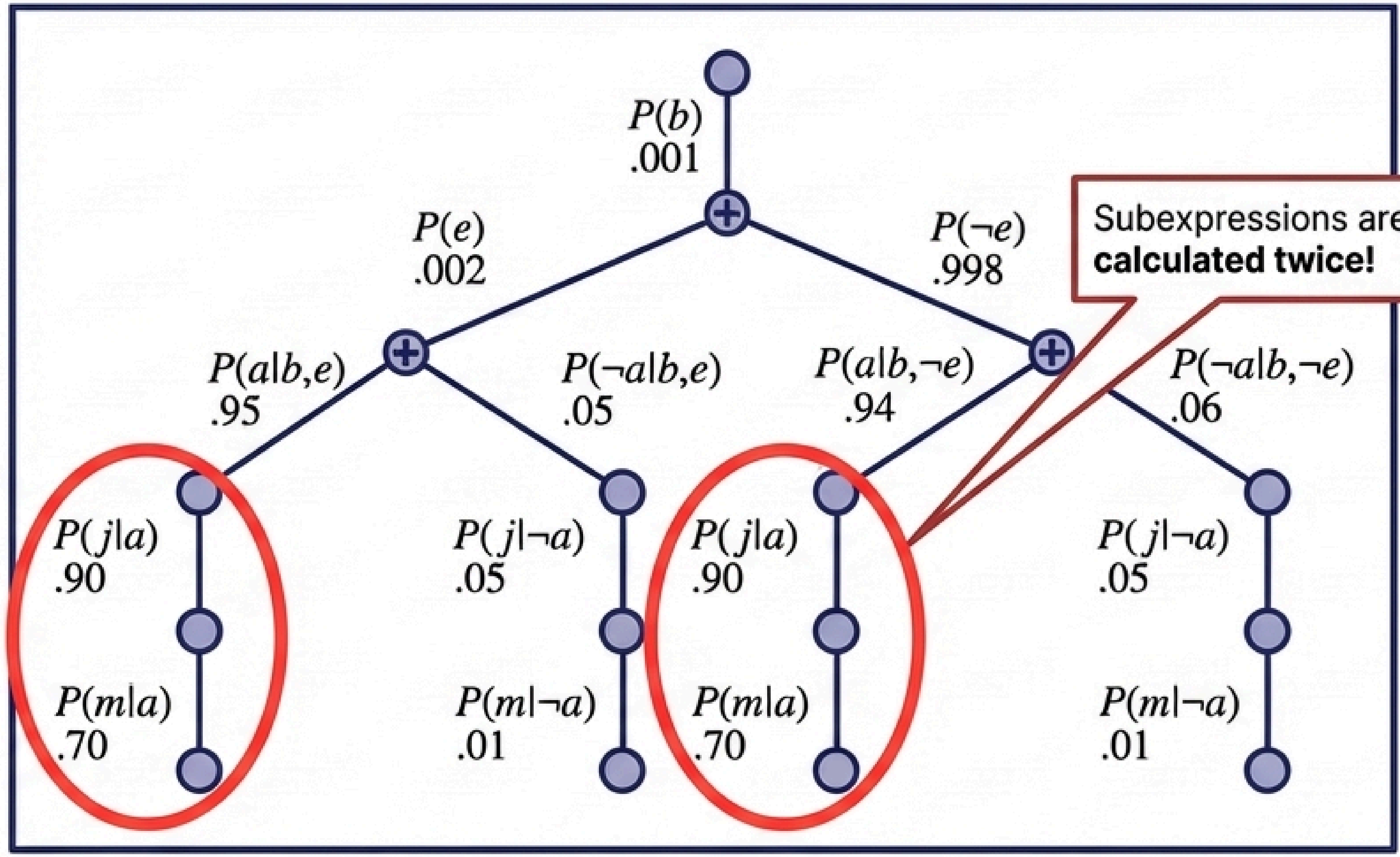
Goal: Compute $P(X|e)$ by summing products of conditional probabilities directly from the joint distribution.

Mechanics:

$$P(X|e) = \alpha \sum_{y} P(X, e, y)$$

Pulls sums inward:

$$P(b|j,m) = \alpha\, P(b) \sum_{e} P(e) \sum_{a} P(a|b,e) P(j|a) P(m|a)$$

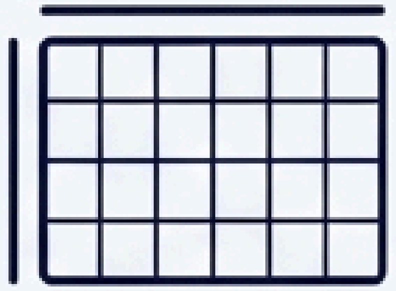


Warning: Time complexity is $O(2^n)$. It operates identically to a naive depth-first search. By failing to store intermediate results, it wastes massive computing power repeating subexpressions.

Exact Inference II: Variable Elimination

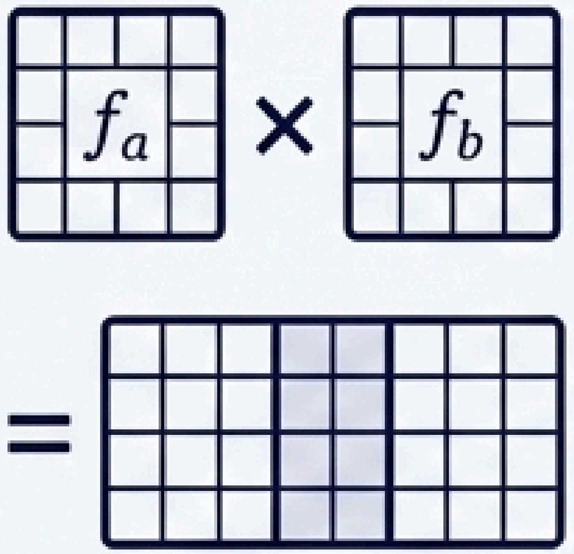
A dynamic programming approach to avoid the Enumeration Trap.

Step 1: Initialize Factors



Convert CPTs of relevant variables into matrices (e.g., f_4(A) for P(j|a)).

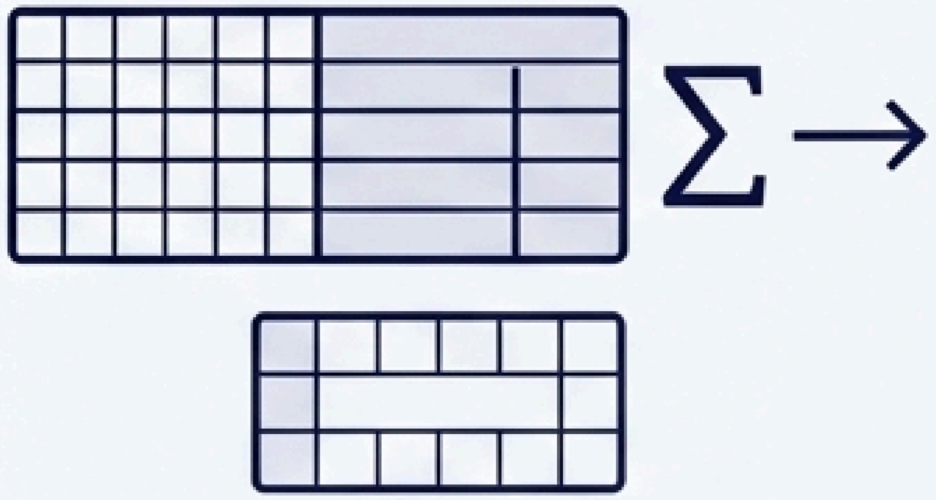
Step 2: Pointwise Multiplication



Multiply interacting factors to create new, combined factors.

Example: $f_3(A,B,E) \times f_4(A) \times f_5(A)$

Step 3: Sum-Out



Eliminate hidden variables by summing across their rows, shrinking the matrix.

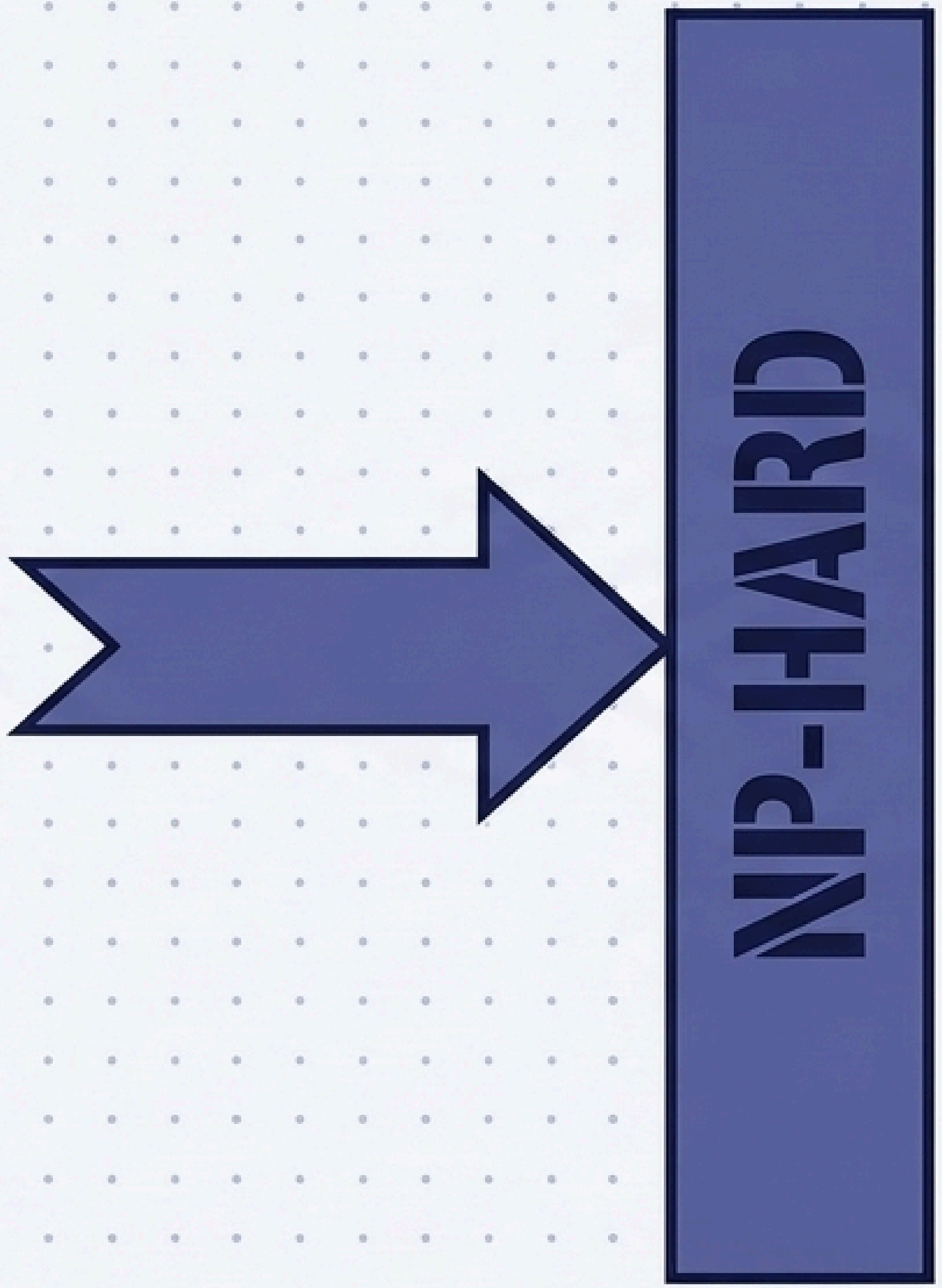
Efficiency Hacks:

- Irrelevant variables (leaf nodes not part of query or evidence) are pruned before calculation begins.
- Complexity depends entirely on the elimination ordering (heuristics choose the variable that minimizes the next factor's size).

The Computational Wall: Why Approximate Inference?

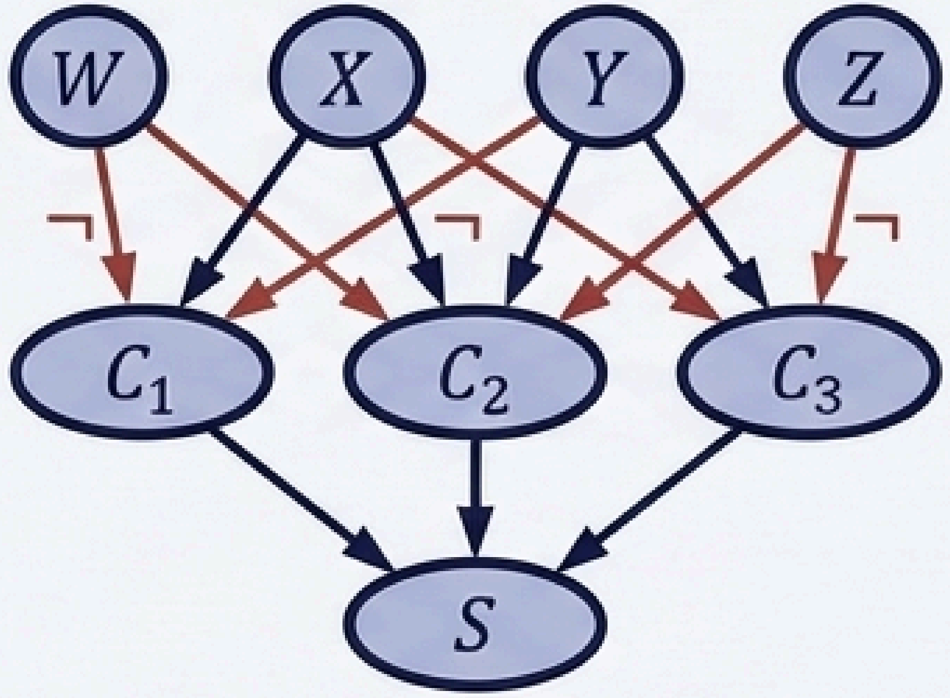
The Limit of Exact Inference

- Variable Elimination is linear $O(n)$ ONLY for "Polytrees" (singly connected networks).
- In multiply connected networks, Variable Elimination can blow up to exponential time and space complexity.



The NP-Hard Proof

- Inference in Bayes Nets includes propositional logic as a special case.
- Any 3-SAT problem can be encoded directly into a Bayes Net (clauses = deterministic disjunctions).



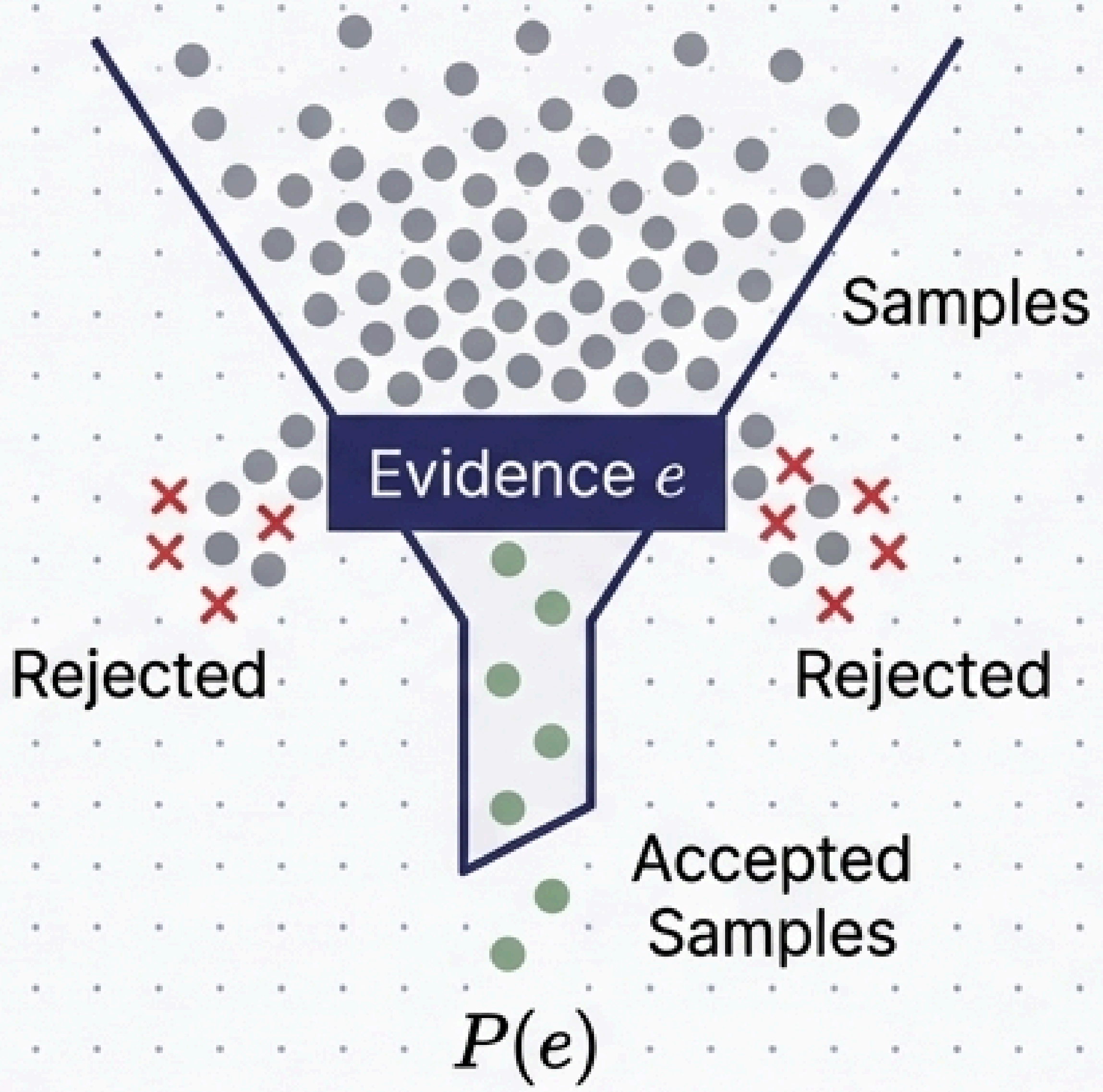
- Thus, Exact Inference is mathematically NP-Hard.

The Pivot: We must sacrifice perfect exactness for computational feasibility → Enter Monte Carlo (Sampling) Algorithms.

Approximate Inference I: Direct & Rejection Sampling

Direct Sampling Baseline

- Sample each variable in topological order conditioned on its parents.
- Produces events drawn entirely from the prior joint distribution.



Answering Queries $P(X|e)$

Step 1: Generate samples using Direct Sampling.

Step 2: Reject (throw away) any sample that contradicts the evidence e.

Step 3: Count occurrences of $X = x$ in the remaining surviving samples.

The Fatal Flaw

The fraction of accepted samples is exactly $P(e)$. As evidence variables grow, the probability of generating a matching sample drops exponentially. Highly inefficient.

Approximate Inference II: Likelihood Weighting & MCMC

Likelihood Weighting

- **Mechanism:** Fixes evidence variables to their observed values. Samples the rest.
- **The Weight:** Because we forced the evidence, the sample isn't natural. We **weight it** by the likelihood of that evidence occurring given the parents:

$$w = \prod P(e_i \mid \text{parents}(E_i))$$

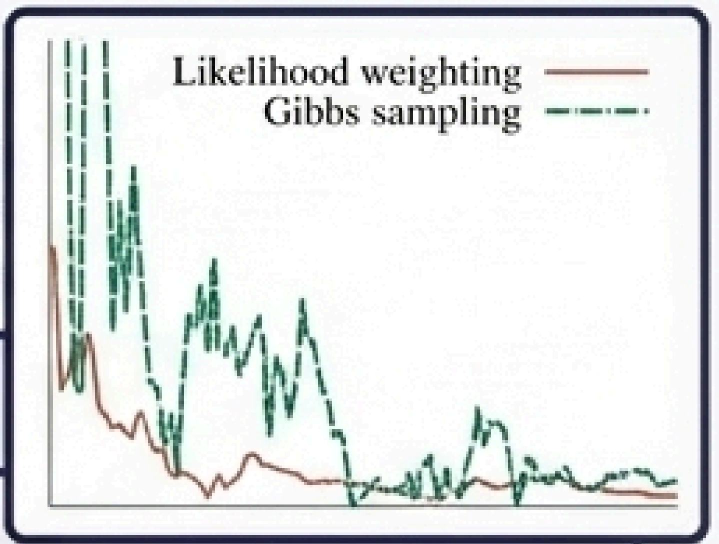
Critique: Better than Rejection, but degrades if evidence is "downstream" (late in ordering), creating hallucinated samples with near-zero weights.

Gibbs Sampling (MCMC)

- **Mechanism:** Starts with a random state. Iteratively picks one non-evidence variable at a time and resamples it based on its Markov Blanket.
- **The Journey:** Wanders the state space via a Markov Chain. Guaranteed to eventually converge to the true posterior distribution (stationary distribution).

Critique: Highly effective, but convergence (mixing rate) can be slow if deterministic rules isolate parts of the state space.

The Ultimate Inference Comparison Matrix



Algorithm	Type	Core Mechanism	Pros & Cons
Enumeration	Exact	Sums all paths	**Pro:** Simple **Con:** $O(2^n)$ time, repeats work
Variable Elimination	Exact	Dynamic Programming (Factors)	**Pro:** Fast for Polytrees **Con:** NP-Hard for multiply connected nets
Rejection Sampling	Approx	Filter prior samples	**Pro:** Consistent **Con:** Unusable for rare evidence
Likelihood Weighting	Approx	Fix evidence & weight	**Pro:** Uses all samples **Con:** Fails on downstream evidence
Gibbs Sampling (MCMC)	Approx	Markov Blanket flipping	**Pro:** Robust state space mapping **Con:** Slow mixing on deterministic links

Takeaway: Real-world probabilistic reasoning demands a tradeoff. Exactness is mathematically elegant but computationally bound; Approximation trades certainty for scale.